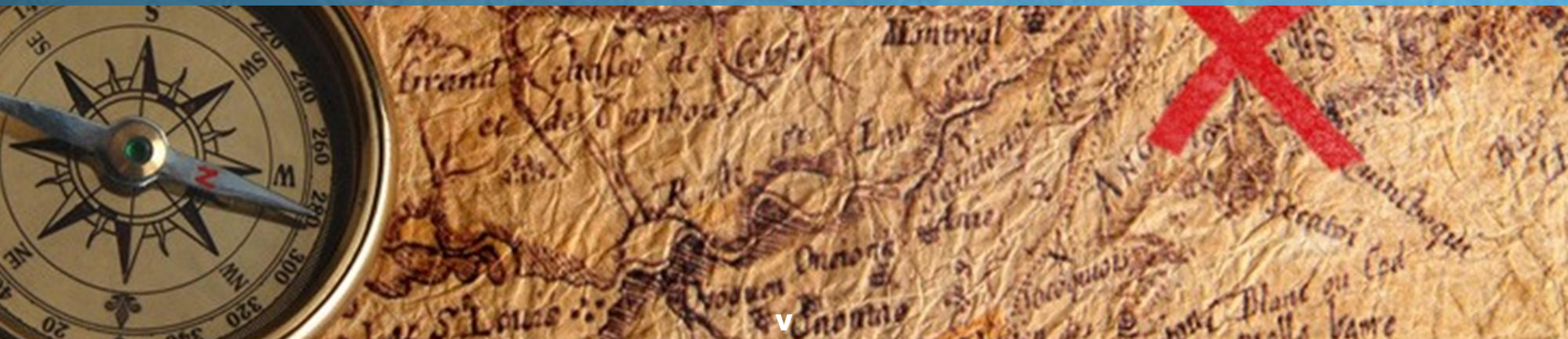


1. Resolución de problemas y algoritmos de búsqueda

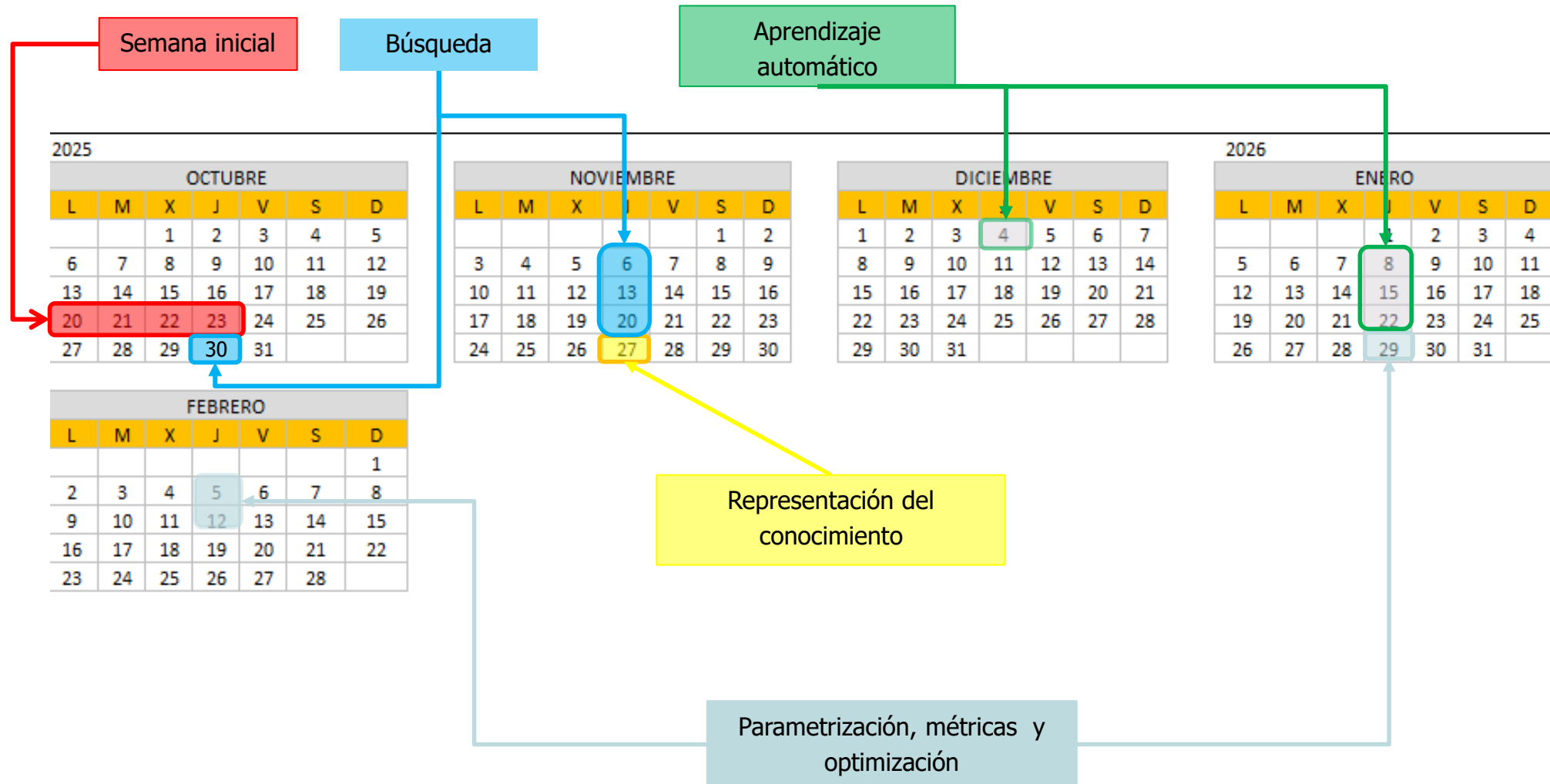


MD005 – Sistemas Basados en el Conocimiento

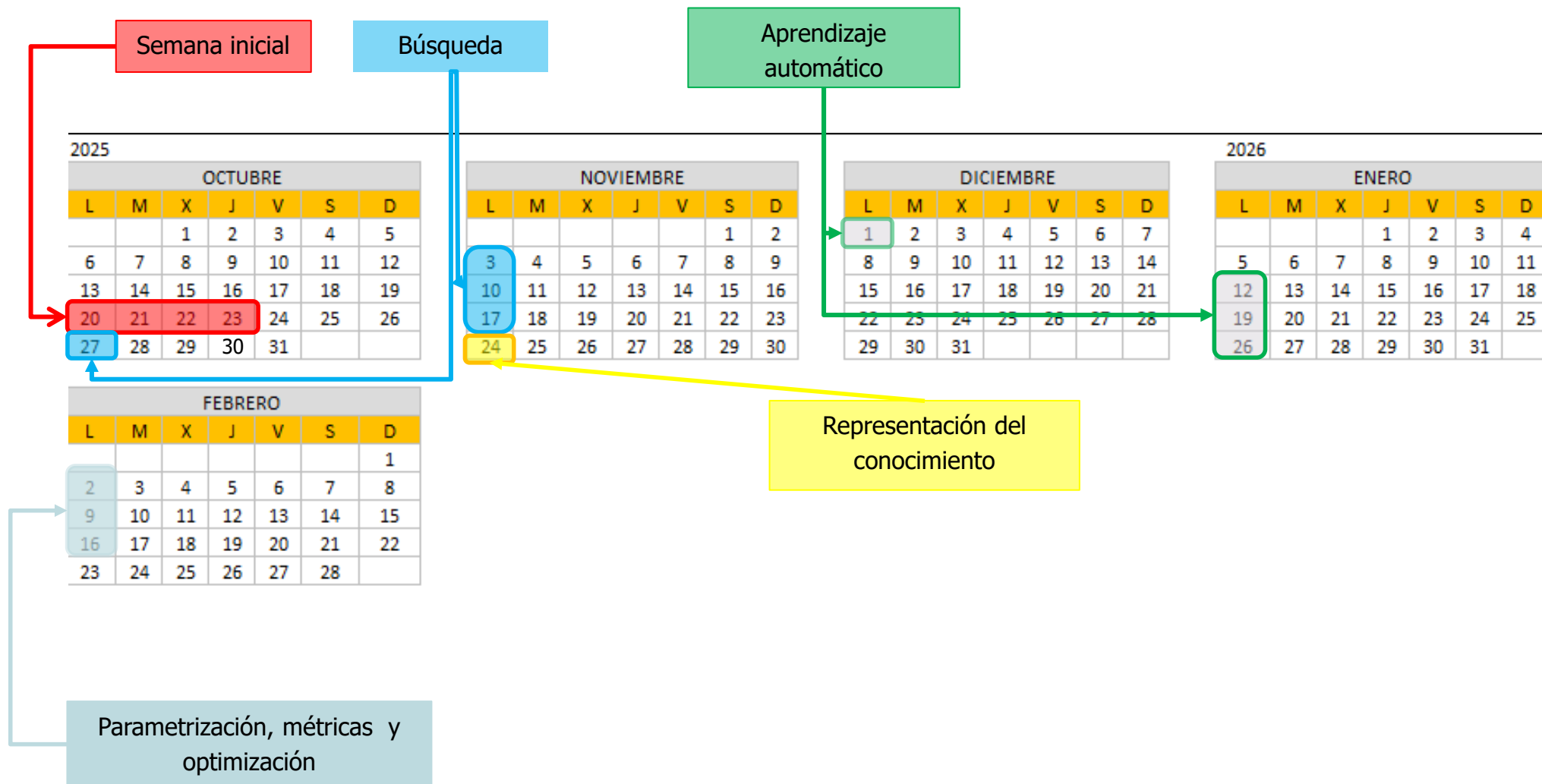
Manel Cerezo García – manel.cerezo@salle.url.edu

“Roadmap” MD005

Organización GRUPO A



Organización GRUPO B



Evaluación

100% Entrega global

- 25% Agrupación
- 25% KNN
- 25% Árboles de decisión
- 25% Optimización/parametrización

Nota válida: Nota individual ≥ 5



Problemas

¿Qué entendemos por problema?

- Definición de problema (diccionario)
 1. m. Cuestión que se trata de aclarar.
 2. m. Proposición o dificultad de solución dudosa.
 3. m. Conjunto de hechos o circunstancias que dificultan la consecución de algún fin.
 4. m. Disgusto, preocupación. U. m. en pl. *Mi hijo solo da problemas.*
 5. m. Planteamiento de una situación cuya respuesta desconocida debe obtenerse a través de métodos científicos.

¿Qué entendemos por problema?

- Definición de problema (diccionario)
 1. m. Cuestión que se trata de aclarar.
 2. m. Proposición o dificultad de solución dudosa.
 3. m. Conjunto de hechos o circunstancias que dificultan la consecución de algún fin.
 4. m. Disgusto, preocupación. U. m. en pl. *Mi hijo solo da problemas.*
 5. m. Planteamiento de una situación cuya respuesta desconocida debe obtenerse a través de métodos científicos.

Situaciones en las que partiremos de una casuística inicial buscando los pasos a seguir para llegar a un resultado que podamos considerar válido u óptimo.

¿Qué entendemos por problema?

- Proverbio Chino

Si un problema tiene solución ¿para qué preocuparse? y si no lo tiene, ¿para qué preocuparse?



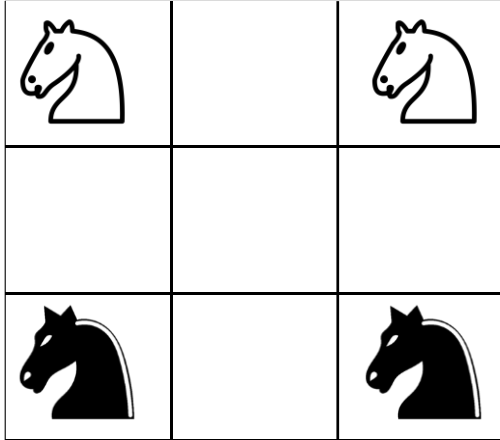
¿Qué entendemos por problema?

- Definición/Identificación de Problema
- Qué genera el problema – Análisis de las causas
- Qué tipo de solución necesita
 - Qué herramienta utilizar
 - Cómo utilizar la herramienta
- Ejecución de un plan
- Evaluación del resultado

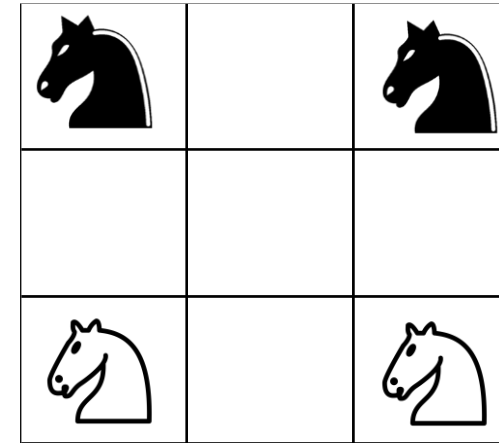


Problema de los 4 caballos

Estado inicial (nodo inicial)



Estado final (nodo objetivo)



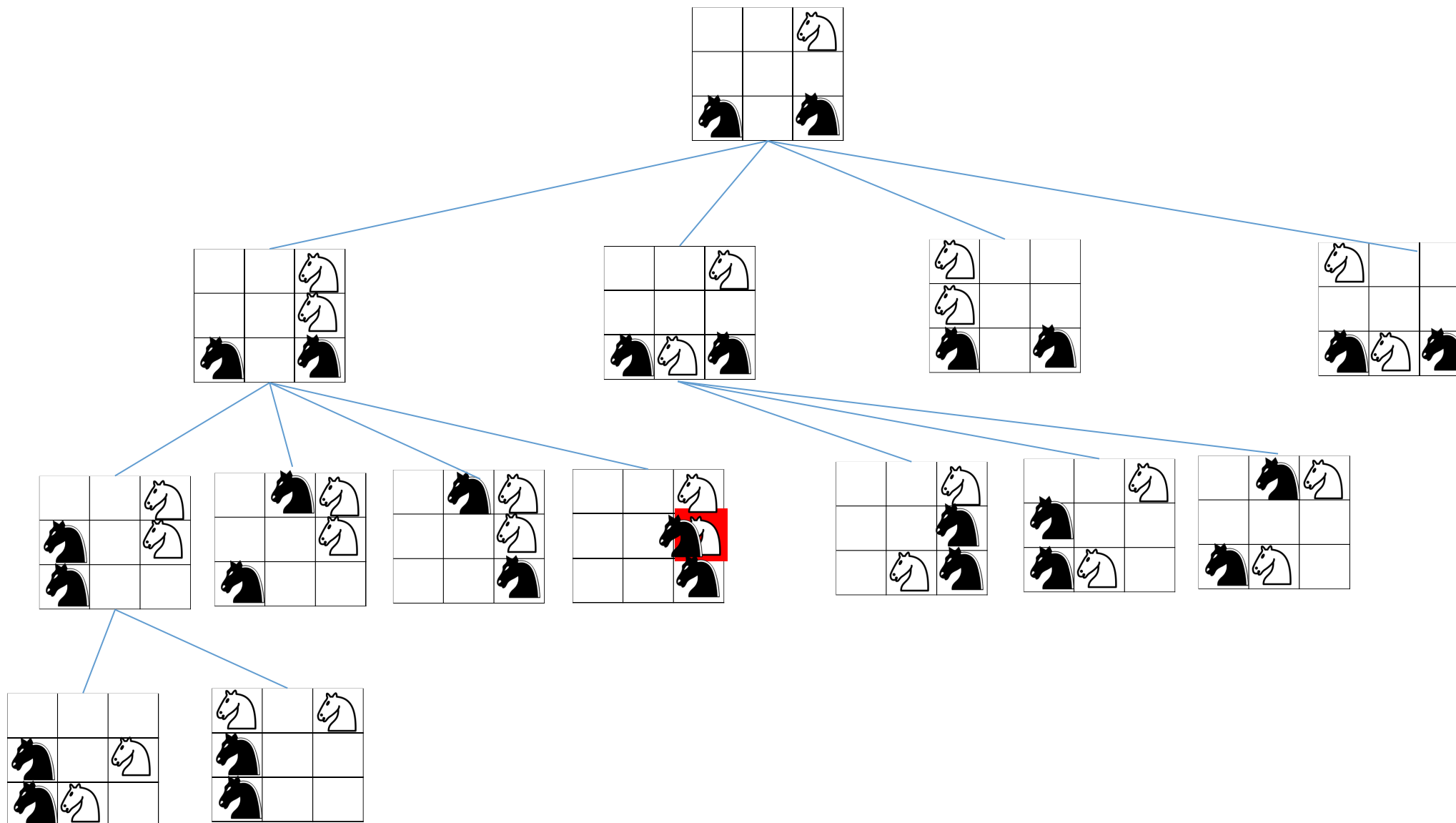
Restricciones

- Los caballos realizan los mismos movimientos que en el ajedrez.
- No puede haber 2 caballos en una misma casilla.
- Las piezas se mueven una a una.

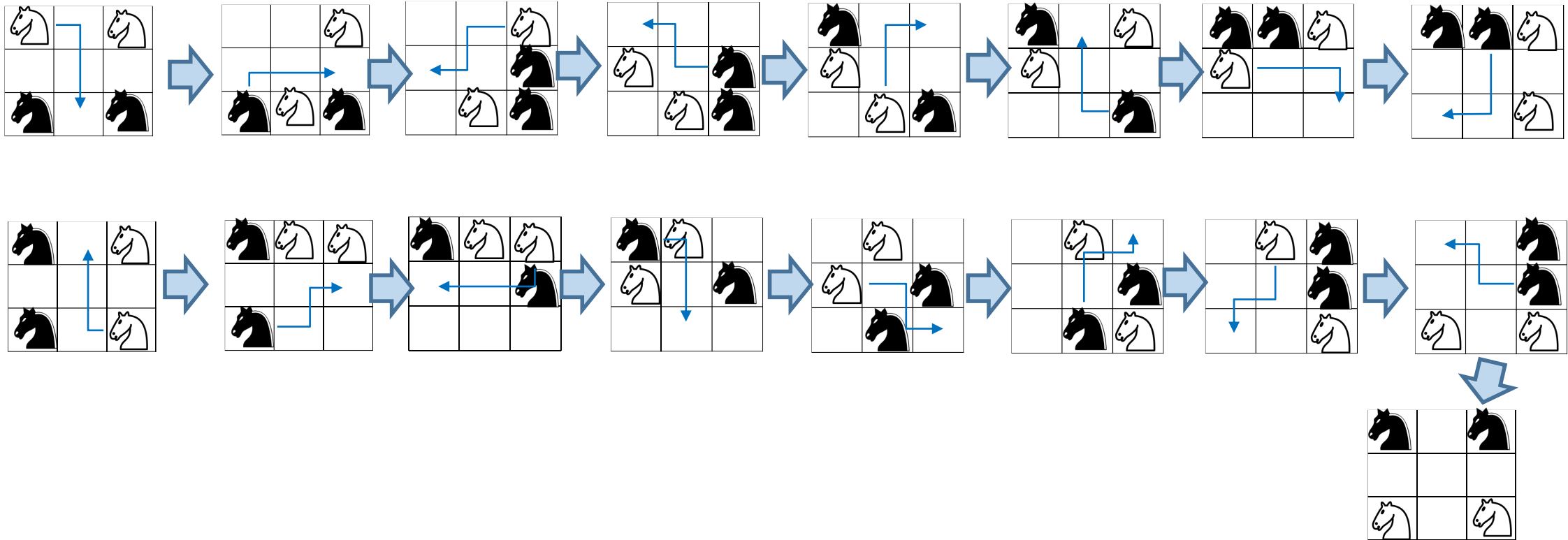
Objetivo

- Encontrar los movimientos que permitan ir del **estado inicial** al **estado final** (= **camino solución**), realizando el menor número de movimientos.

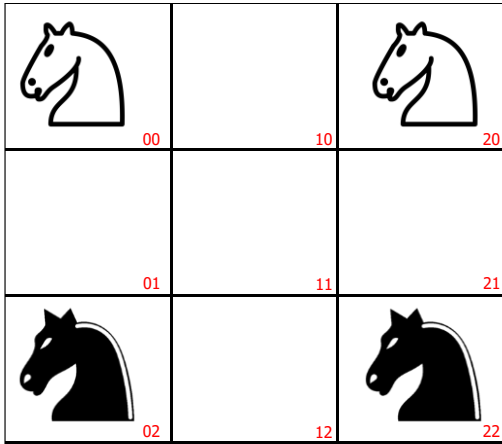
Problema de los 4 caballos - Árbol



Problema de los 4 caballos – solución 1



Problema de los 4 caballos - Representación





Primer problema

Problema: ¿Cómo llegamos a clase?



Problema: ¿Cómo llegamos a clase?



1. Puertas de cristal en el vestíbulo.
2. Bajamos a la planta inferior.
3. Cruzamos la pista de básquet
4. Entramos al edificio Sant Jaume Hilari
5. Subimos al piso 0



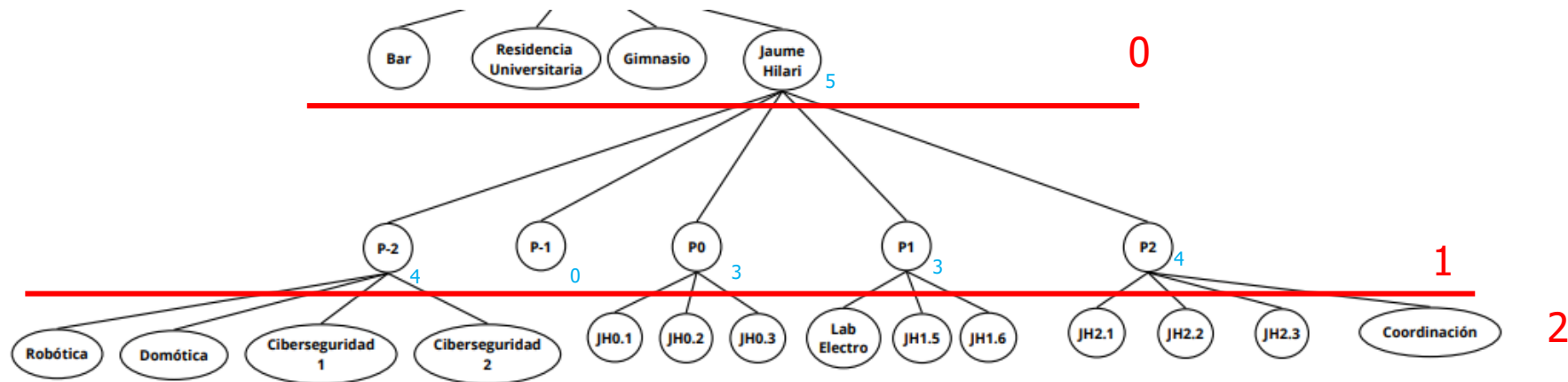
Resolución de problemas Conceptos

Conceptos

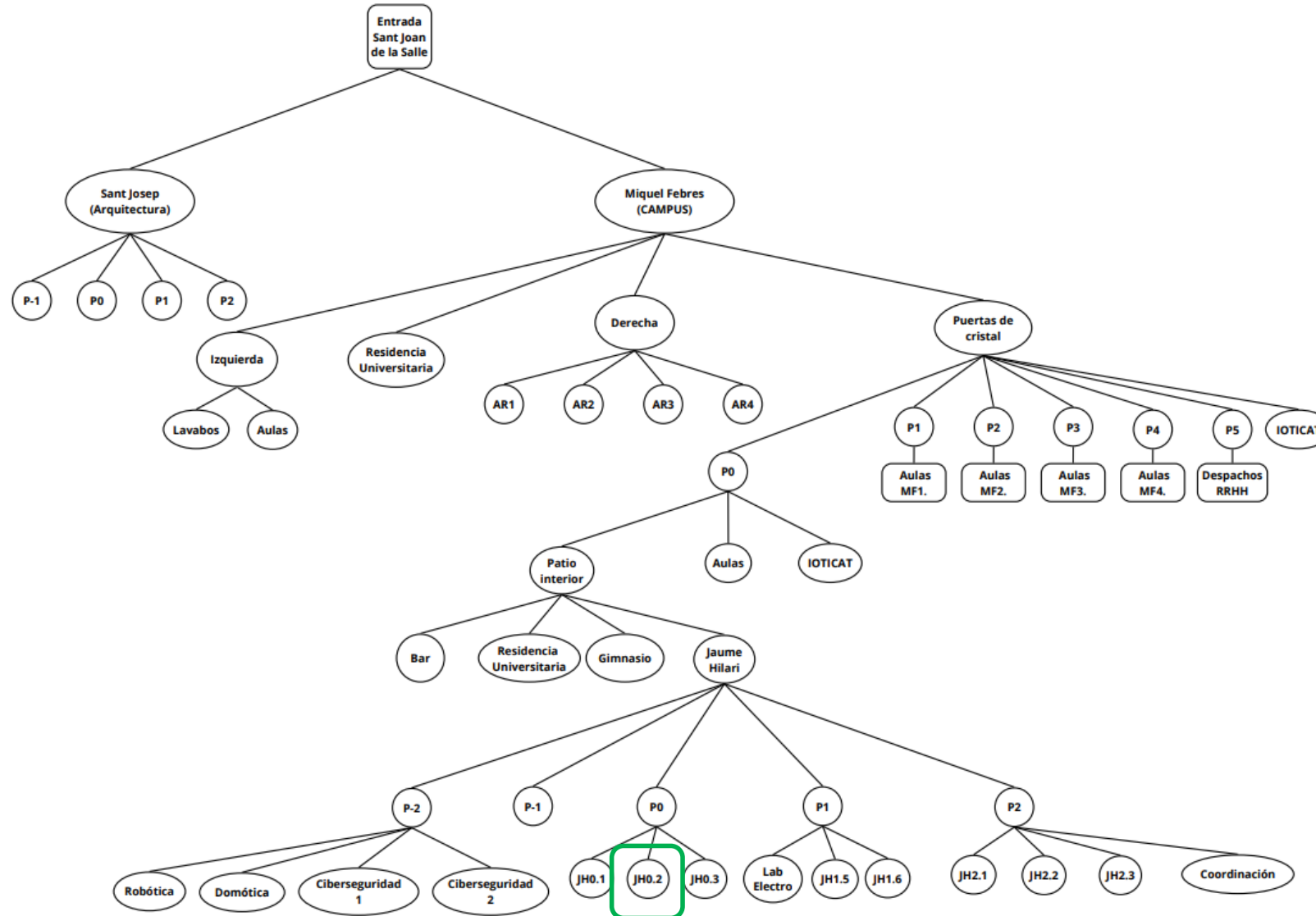
- **Espacio de búsqueda:** Conjunto de situaciones/combinaciones posibles dentro del problema.
- **Objetivo:** Encontrar un camino dentro del espacio de búsqueda que conecte el **nodo inicial** con el **nodo objetivo** dando así la solución del problema.
- **Profundidad:** Nivel máximo al que se puede acceder en el espacio de búsqueda.
- **Factor de ramificación:** Cantidad de caminos posibles en cada nodo.

Conceptos

- **Espacio de búsqueda:** Conjunto de situaciones/combinaciones posibles dentro del problema.
- **Objetivo:** Encontrar un camino dentro del espacio de búsqueda que conecte el **nodo inicial** con el **nodo objetivo** dando así la solución del problema.
- **Profundidad:** Nivel máximo al que se puede acceder en el espacio de búsqueda.
- **Factor de ramificación:** Cantidad de caminos posibles en cada nodo.



Problema: ¿Cómo llegamos a clase?



Objetivo:

JH0.2

Profundidad: 8

Factor de
ramificación:

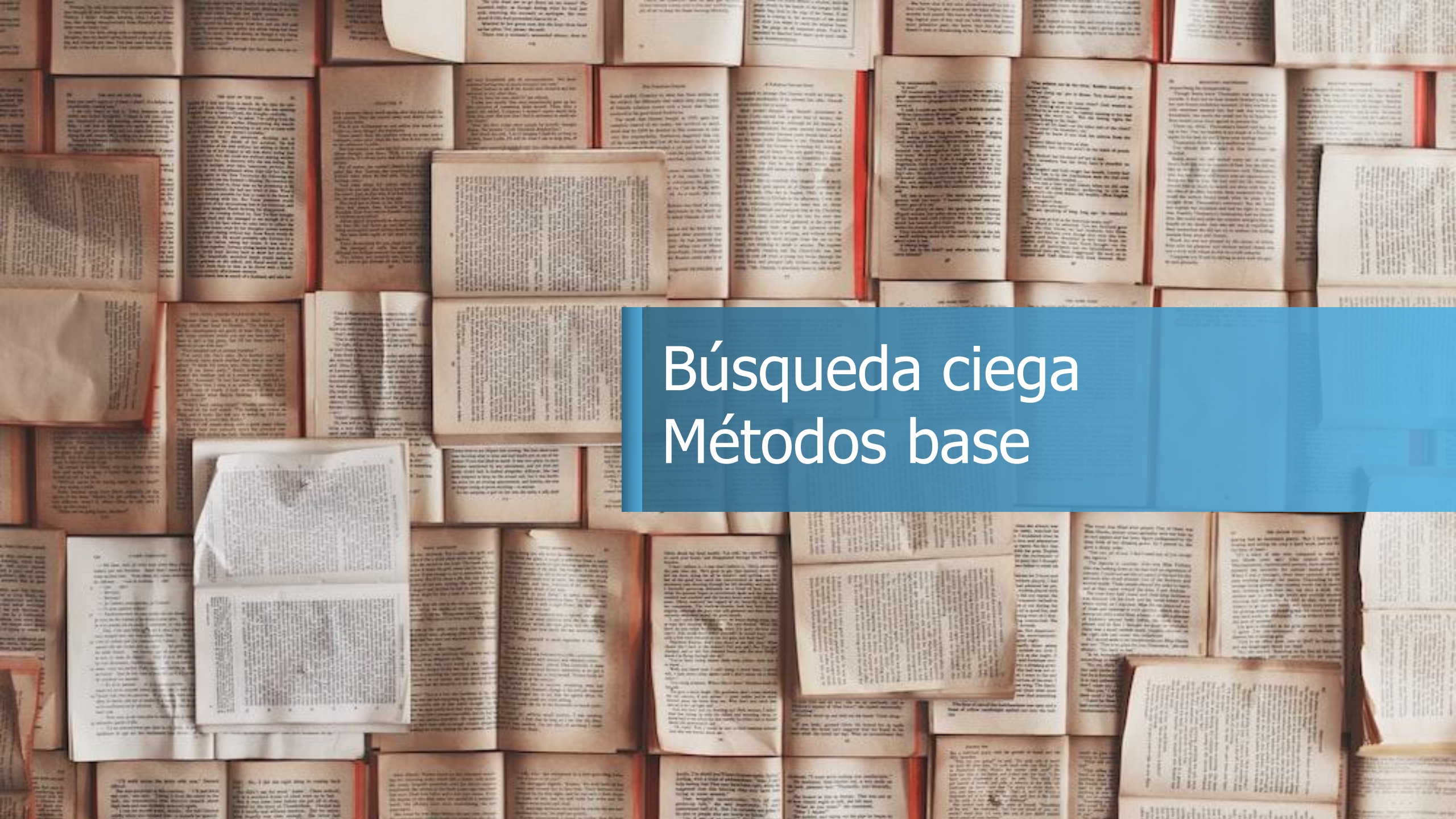
2/4/7/3

Problema: ¿Cómo llegamos a clase?



Objetivo: Llegar a Torrassa desde Sant Roc.

(Solo tener en cuenta el primer intercambio entre líneas)



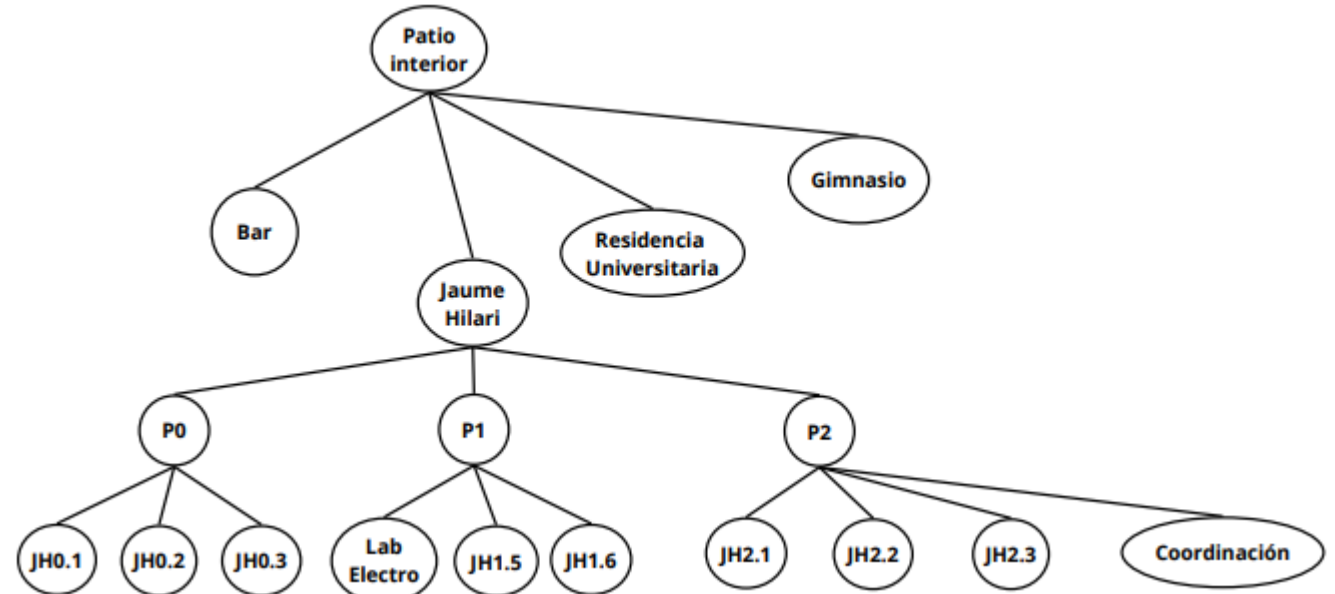
Búsqueda ciega Métodos base

Búsqueda ciega – métodos base

- Tipo de búsqueda sin conocimiento del entorno que intenta encontrar una salida a partir de aplicar de forma consistentes las mismas reglas básicas, no garantizan encontrar la mejor solución posible y a veces, no serán capaces de encontrar una solución.
- Depth First Search (Búsqueda en profundidad)
- Breadth First Search (Búsqueda en amplitud)
- Iterative Deepening Search (Profundidad iterativa)
- Iterative Broadening Search (Amplitud iterativa)

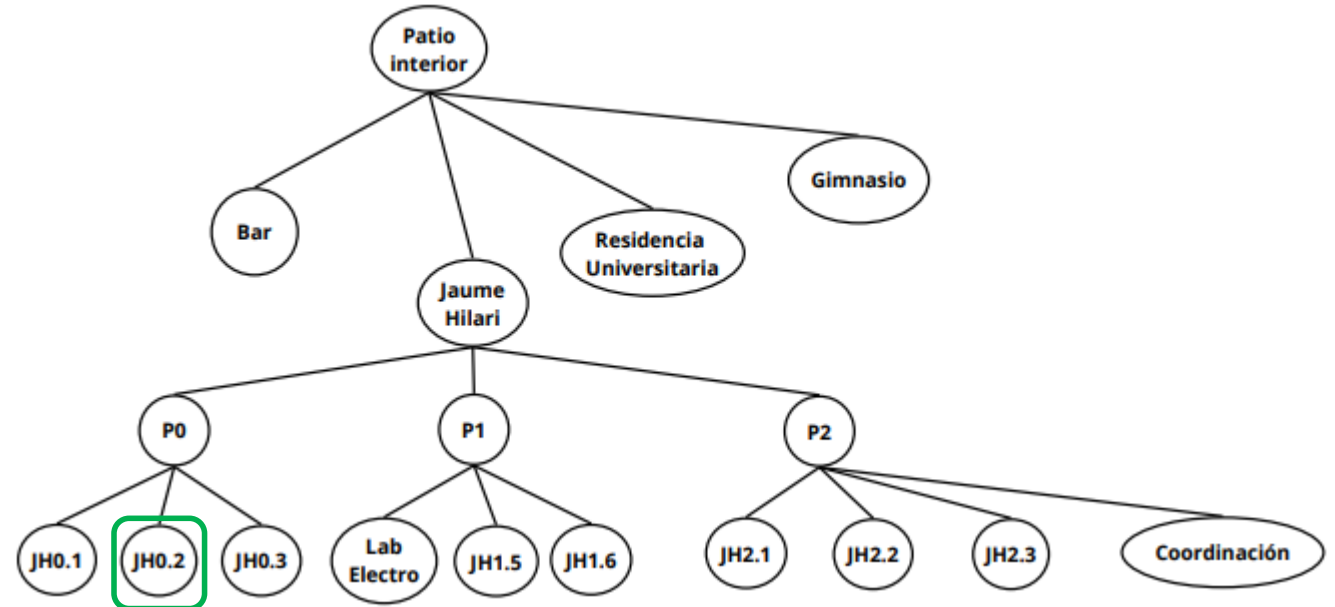
Depth First Search (DFS)

- Busca en profundidad, busca nuevos nodos en el siguiente nivel antes de mantenerse en el mismo.
- ¿Problemas?
 - Cuesta mucho encontrar la solución si se encuentra en los niveles superiores pero en una rama diferente a la que se encuentra.
 - Si hay profundidad infinita no es capaz de encontrar la solución.



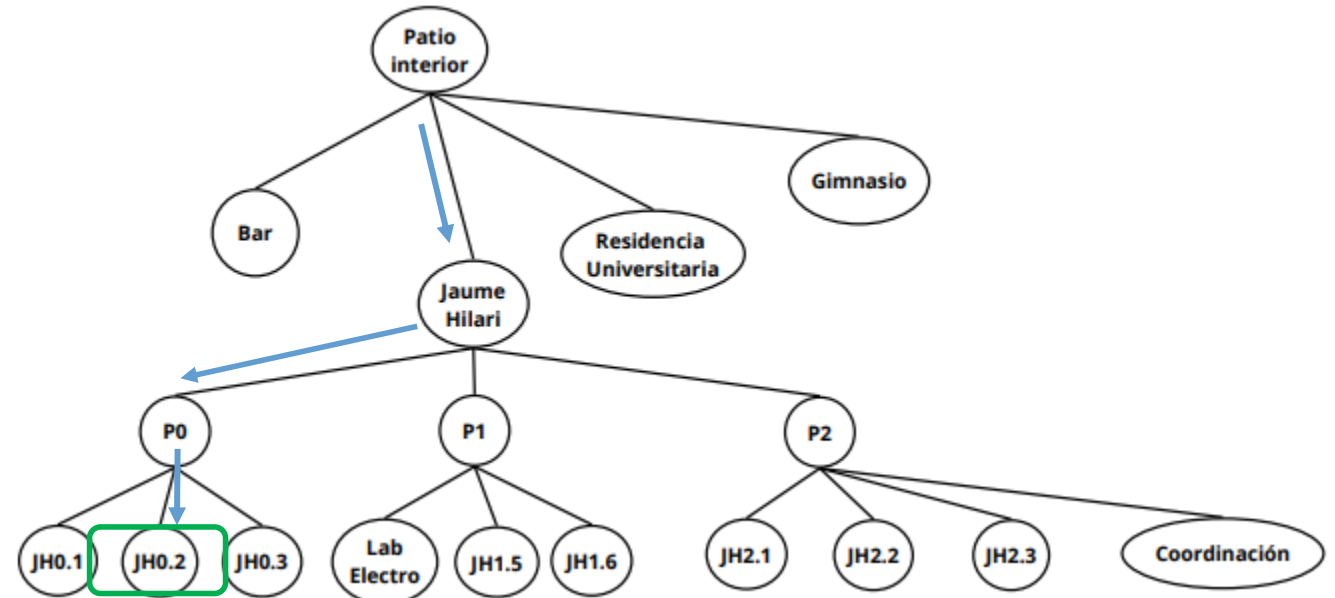
Depth First Search (DFS)

- Busca en profundidad, busca nuevos nodos en el siguiente nivel antes de mantenerse en el mismo.
- ¿Problemas?
 - Cuesta mucho encontrar la solución si se encuentra en los niveles superiores pero en una rama diferente a la que se encuentra.
 - Si hay profundidad infinita no es capaz de encontrar la solución.

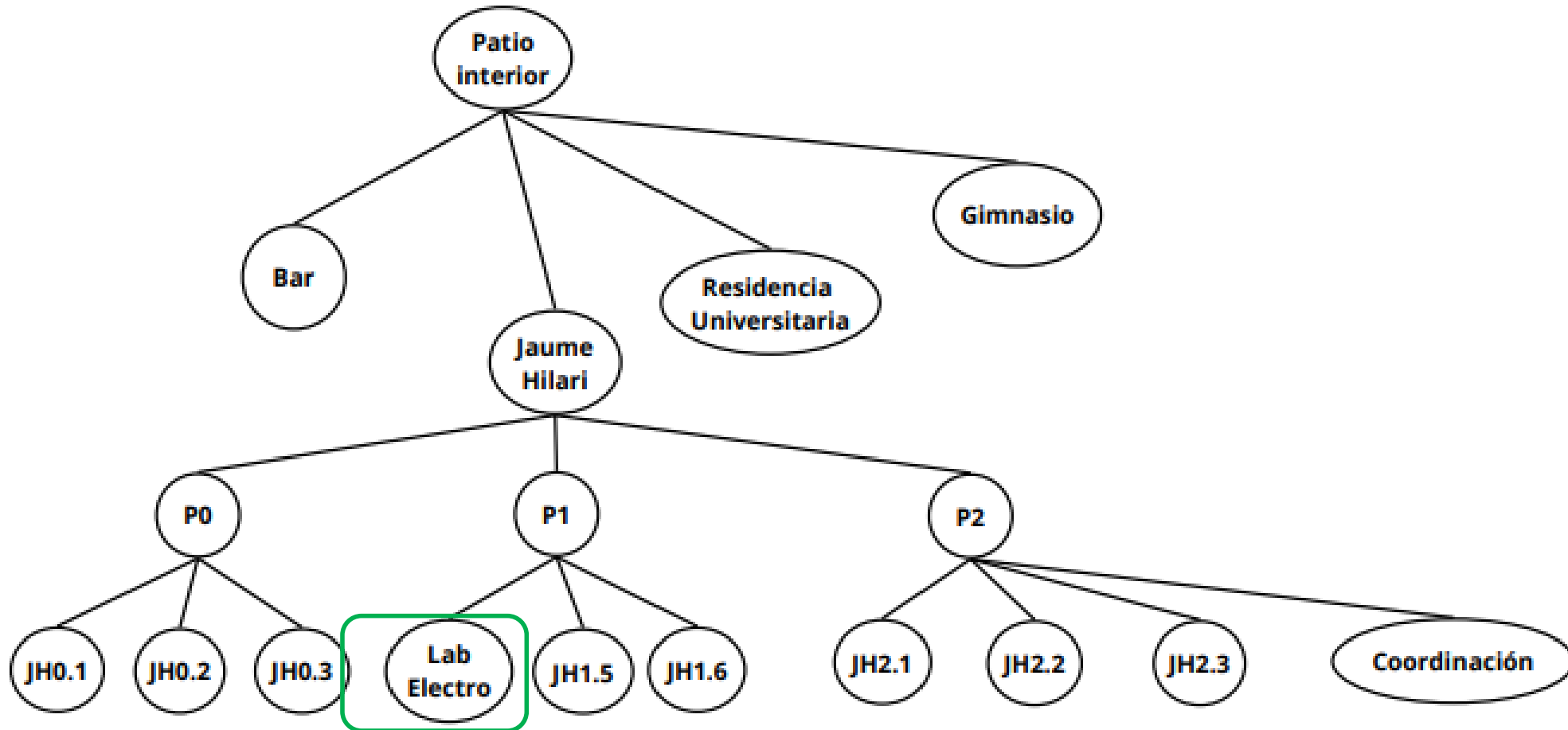


Depth First Search (DFS)

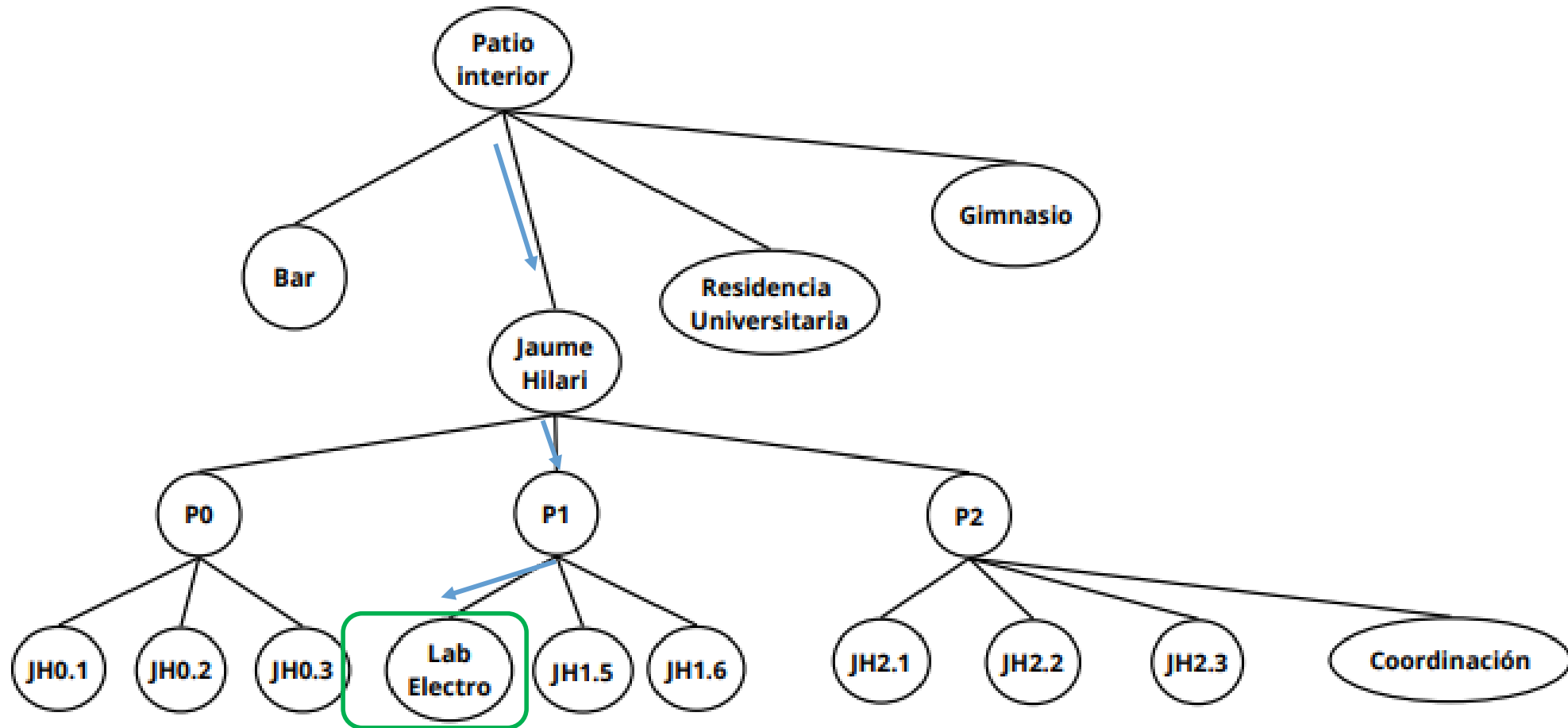
- Busca en profundidad, busca nuevos nodos en el siguiente nivel antes de mantenerse en el mismo.
- ¿Problemas?
 - Cuesta mucho encontrar la solución si se encuentra en los niveles superiores pero en una rama diferente a la que se encuentra.
 - Si hay profundidad infinita no es capaz de encontrar la solución.



Depth First Search (DFS)



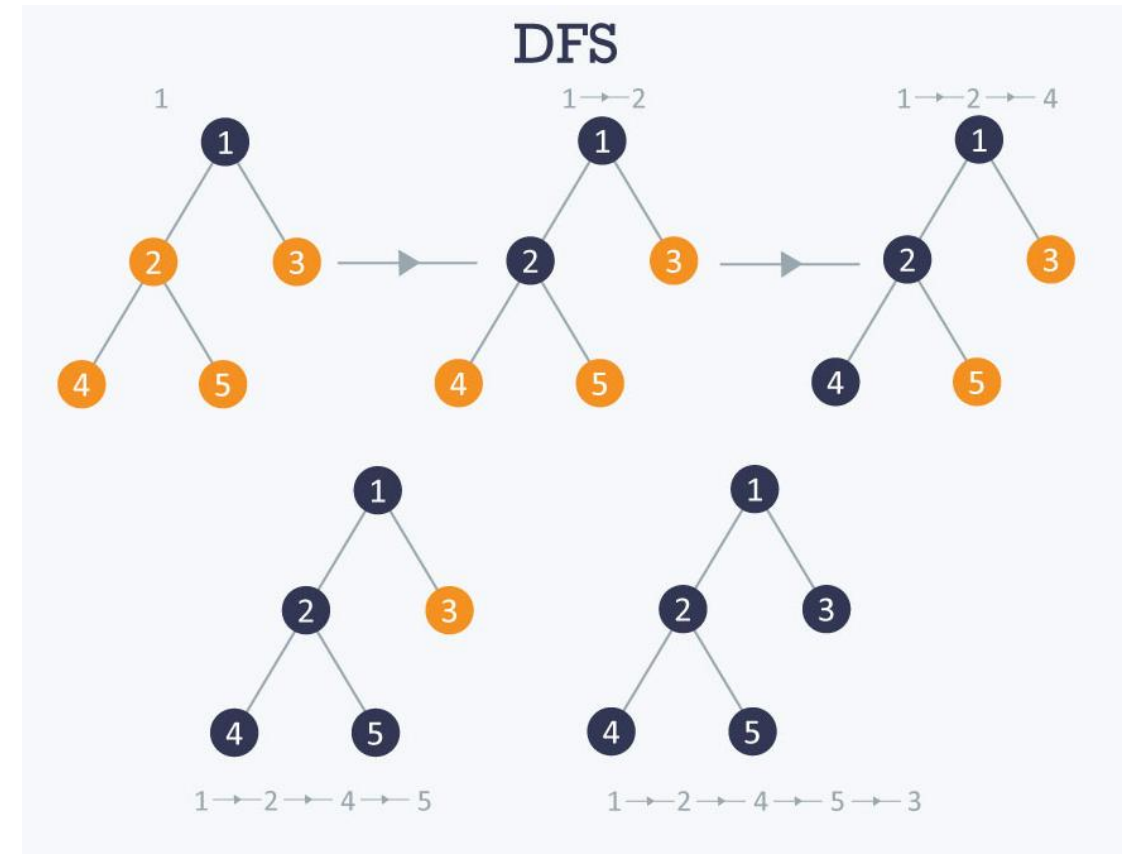
Depth First Search (DFS)



Depth First Search (DFS)

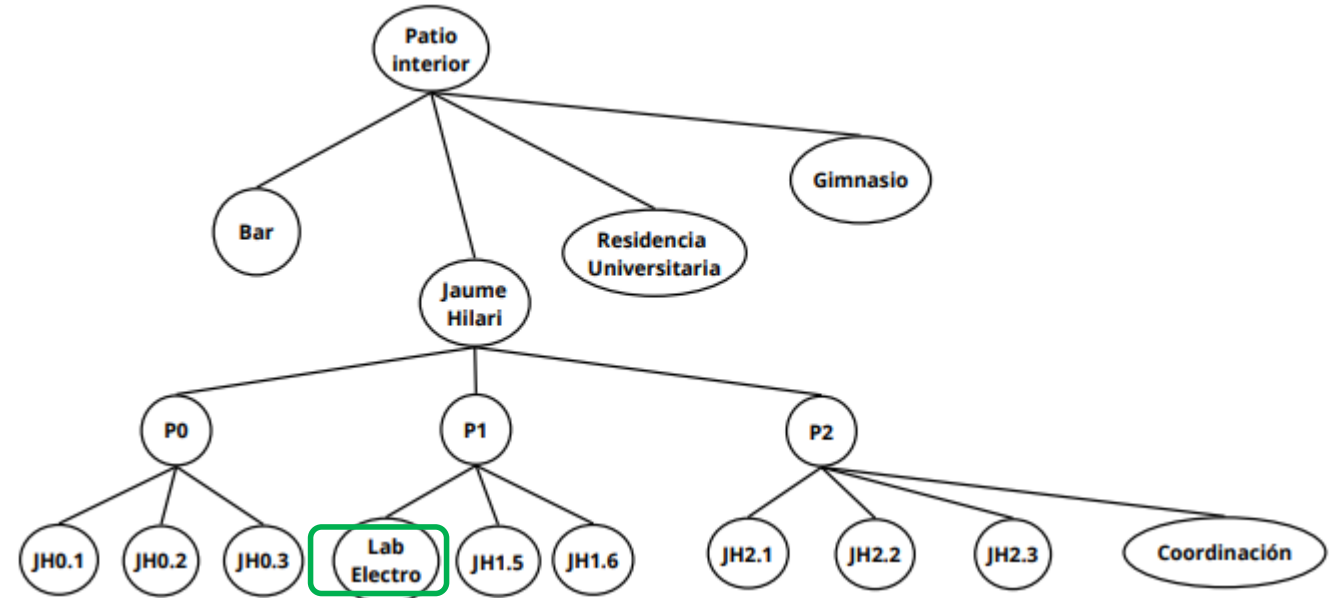
Representaremos los nodos del problema en la lista L e iteraremos con los pasos siguientes:

1. Si L está vacía terminaremos. En caso contrario, escogeremos el primer nodo n.
2. Si n es objetivo paramos y retornamos el camino nodo inicial – nodo objetivo.
3. Si n no es objetivo, lo eliminamos de L.
4. Añadimos los hijos al **principio** de la lista y reiniciamos el ciclo.



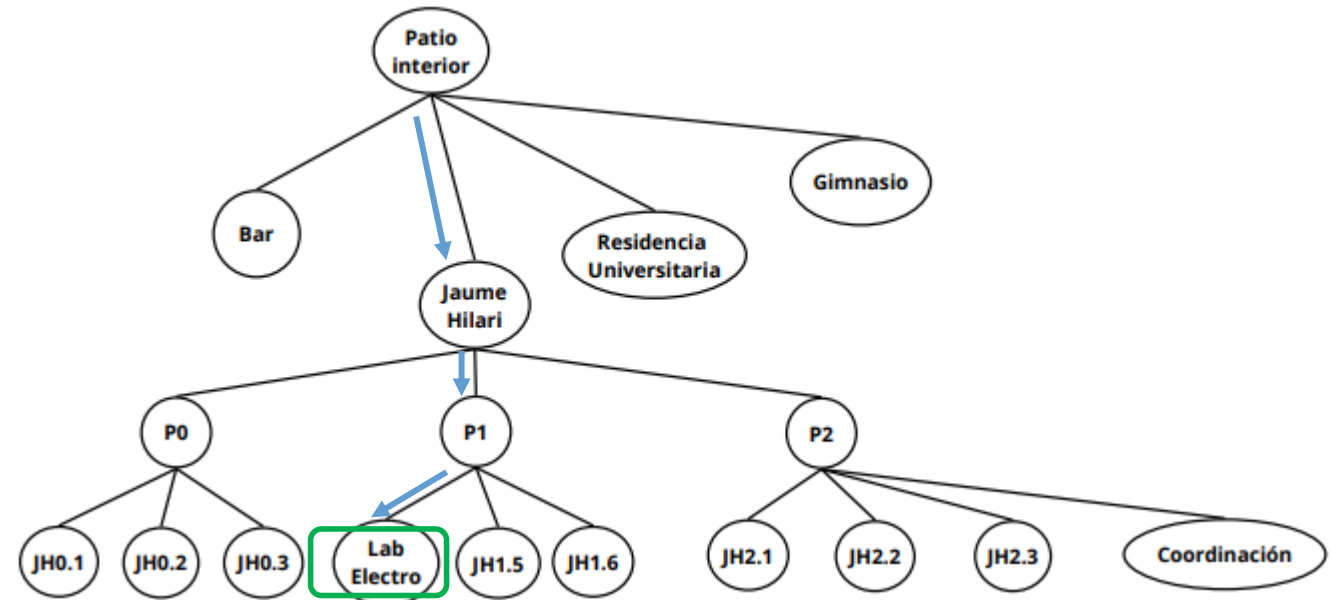
Breadth First Search (BFS)

- Busca en amplitud, prefiriendo los nodos del mismo nivel.
- ¿Problemas?
 - Si la solución se encuentra en una de las hojas será extremadamente lento.
 - Consumo de memoria exponencial al mantener todo el árbol en activo.



Breadth First Search (BFS)

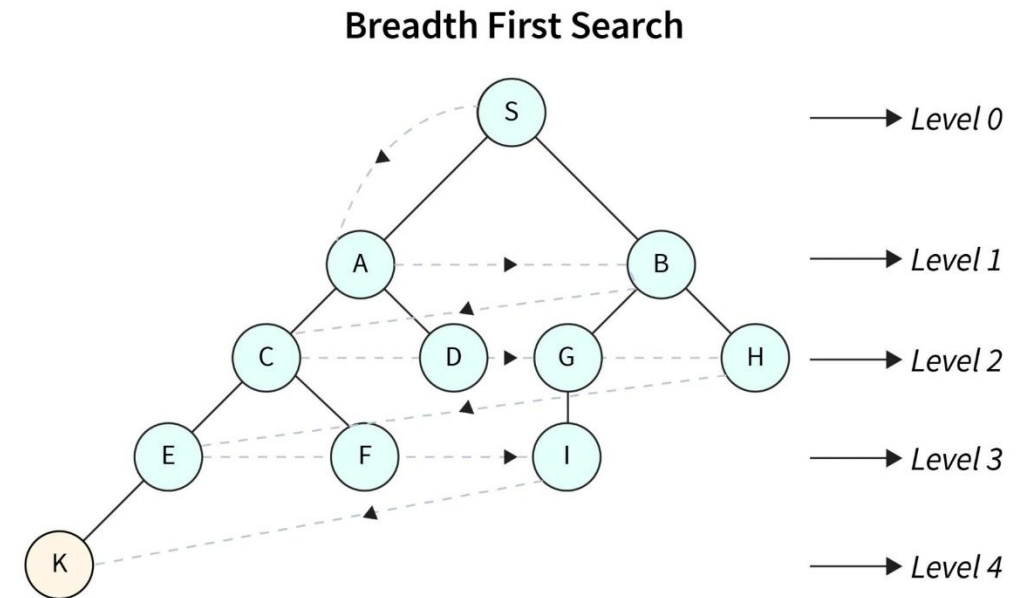
- Busca en amplitud, prefiriendo los nodos del mismo nivel.
- ¿Problemas?
 - Si la solución se encuentra en una de las hojas será extremadamente lento.
 - Consumo de memoria exponencial al mantener todo el árbol en activo.



Breadth First Search (BFS)

Representaremos los nodos del problema en la lista L e iteraremos con los pasos siguientes:

1. Si L está vacía terminaremos. En caso contrario, escogeremos el primer nodo n.
2. Si n es objetivo paramos y retornamos el camino nodo inicial – nodo objetivo.
3. Si n no es objetivo, lo eliminamos de L.
4. Añadimos los hijos al **final** de la lista y reiniciamos el ciclo.

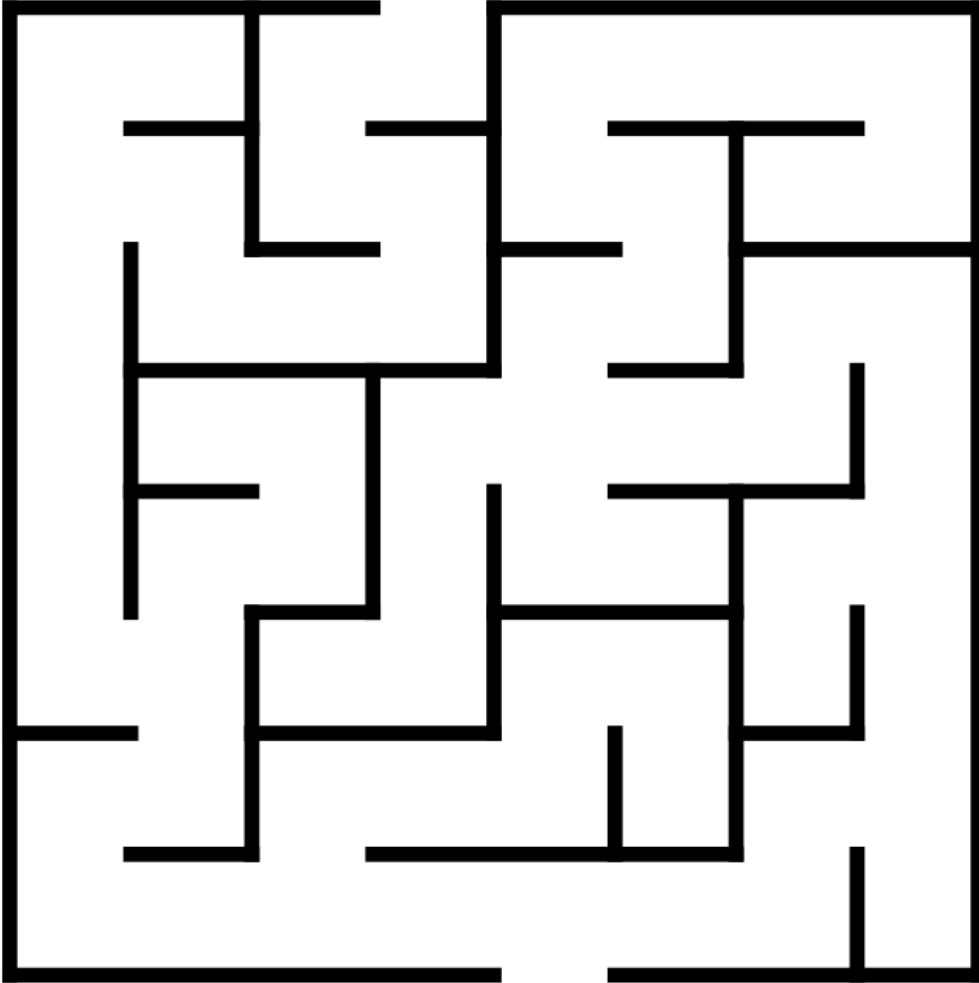


DFS & BFS

	DFS	BFS
Costes memoria	$d(b - 1)$	b^{d-1}
Costes tiempo	$\frac{b^d}{2}$	$\frac{b^d}{2}$
Problemas	- Cuesta mucho encontrar la solución si se encuentra en los niveles superiores pero en una rama diferente a la que se encuentra. - Si hay profundidad infinita no es capaz de encontrar la solución.	- Si la solución se encuentra en una de las hojas será extremadamente lento. - Consumo de memoria exponencial al mantener todo el árbol en la propia memoria.

¿Con cuál nos quedamos?

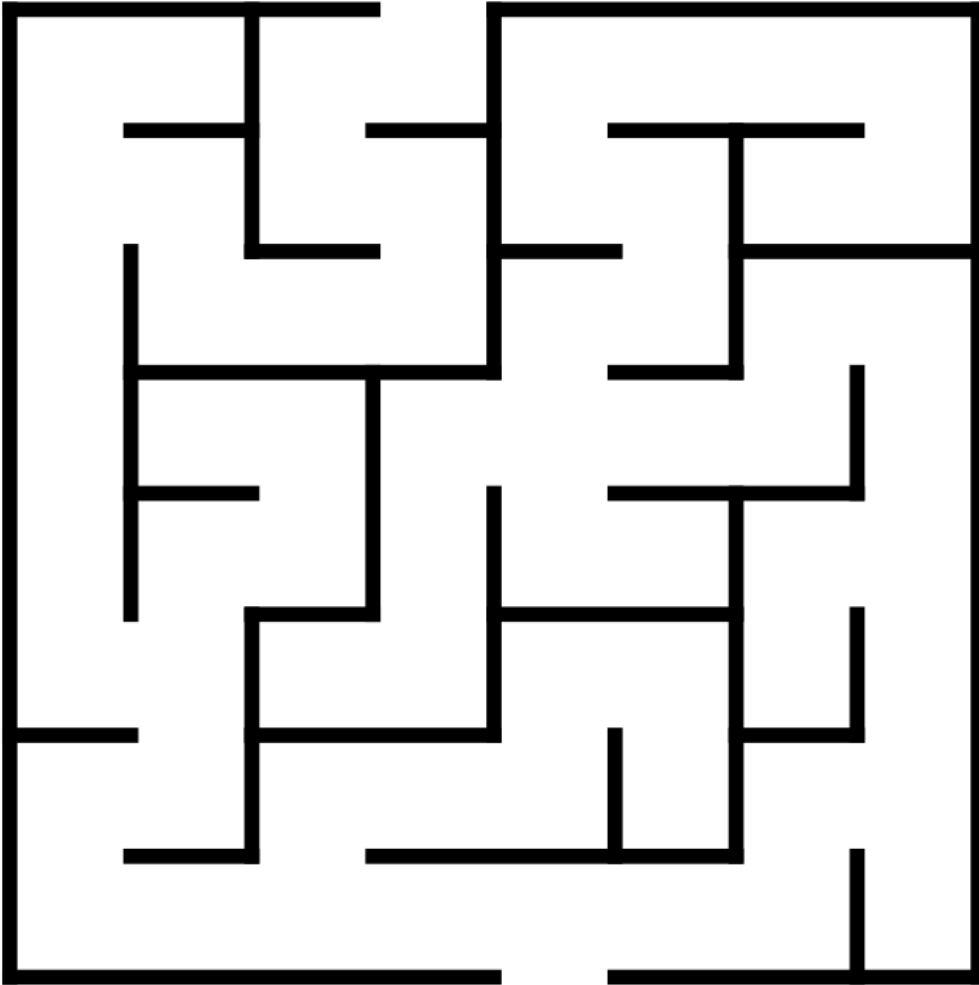
Ejercicio de clase

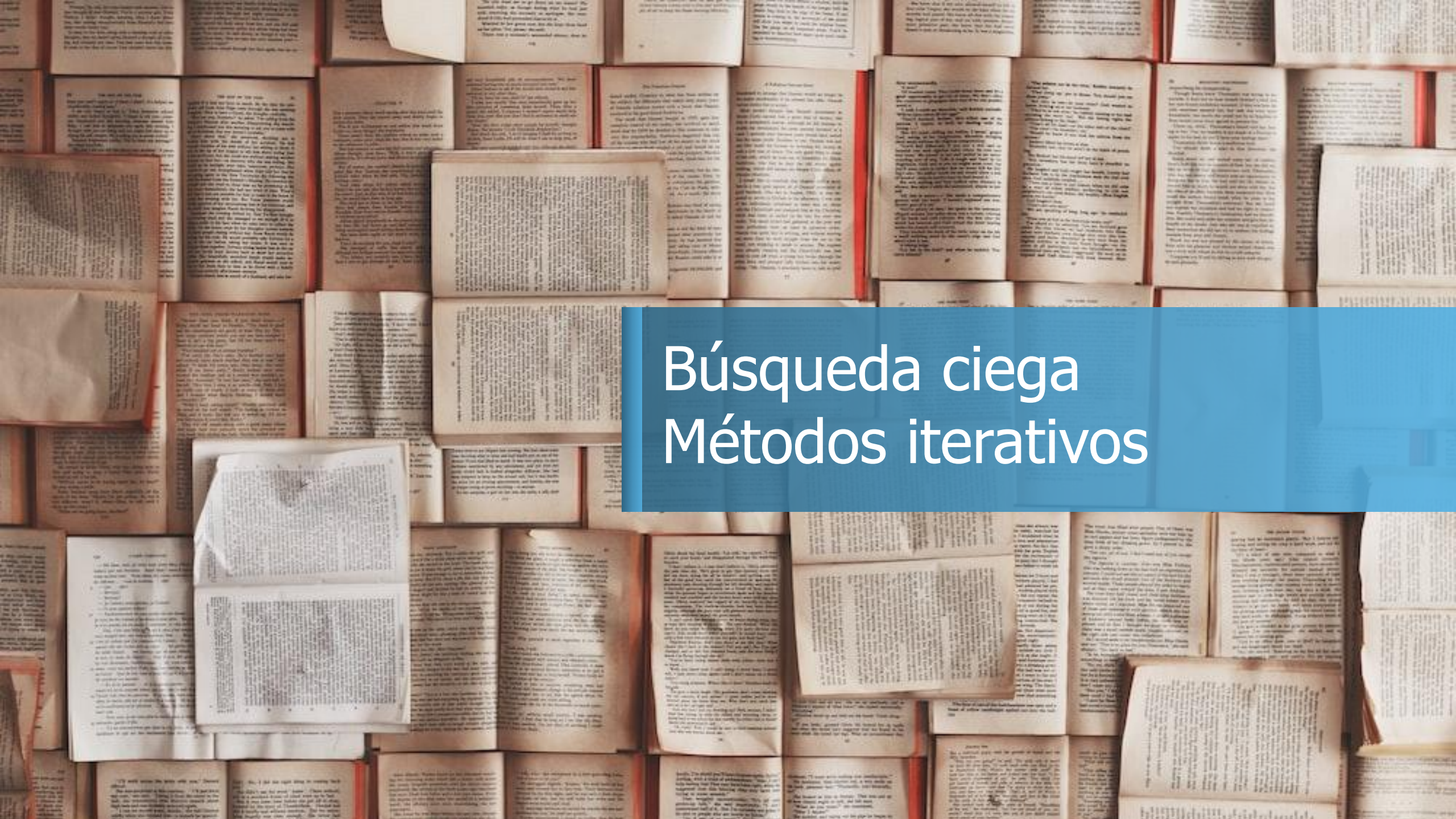


Transformad a esquema de árbol y proporcionar las variables que describen el árbol/problema.

- ¿Ramificación?
- ¿Profundidad?
- Camino hacia la salida.

Ejercicio de clase





Búsqueda ciega Métodos iterativos

Búsqueda ciega – métodos iterativos

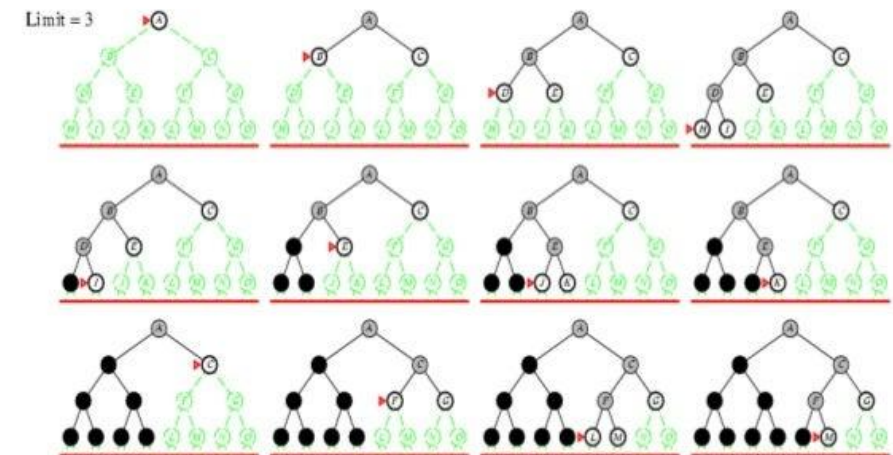
- ¿Qué són?
- Introduce el concepto de profundidad de corte artificial (artificial Depth cutoff). Aprovechan los métodos iniciales (DFS/BFS) y simplifican el espacio de búsqueda con el cutoff evitando los problemas principales de los algoritmos base.

Iterative Deeping Search (IDS)

Representaremos los nodos del problema en la lista L e iteraremos con los pasos siguientes:

1. Sea c el valor artificial de la profundidad de corte (artificial depth cutoff; profundidad en la que se parará la búsqueda DFS). Inicialmente el valor es 1.
2. Sea L una lista con los nodos iniciales del problema.
3. Si L está vacía y $c=d$ entonces terminar.
4. Si L está vacía pero $c < d$ entonces incrementar c y volver al paso 2 (el algoritmo vuelve a empezar con una c diferente).
5. Si no, sea n el primer nodo de la lista L.
6. Si n es el nodo objetivo entonces parar y retornarlo conjuntamente con el camino desde el nodo inicial hasta él.
7. Si no, eliminar n de L.
8. Si la profundidad de n es más pequeña que c ($d(n) < c$) entonces expandir n añadiendo todos sus hijos al principio de L. Cada hijo estará etiquetado con el camino recorrido desde el nodo inicial hasta él.
9. Volver a 3.

Iterative deepening search / =3



Iterative Broadening Search (IBS)

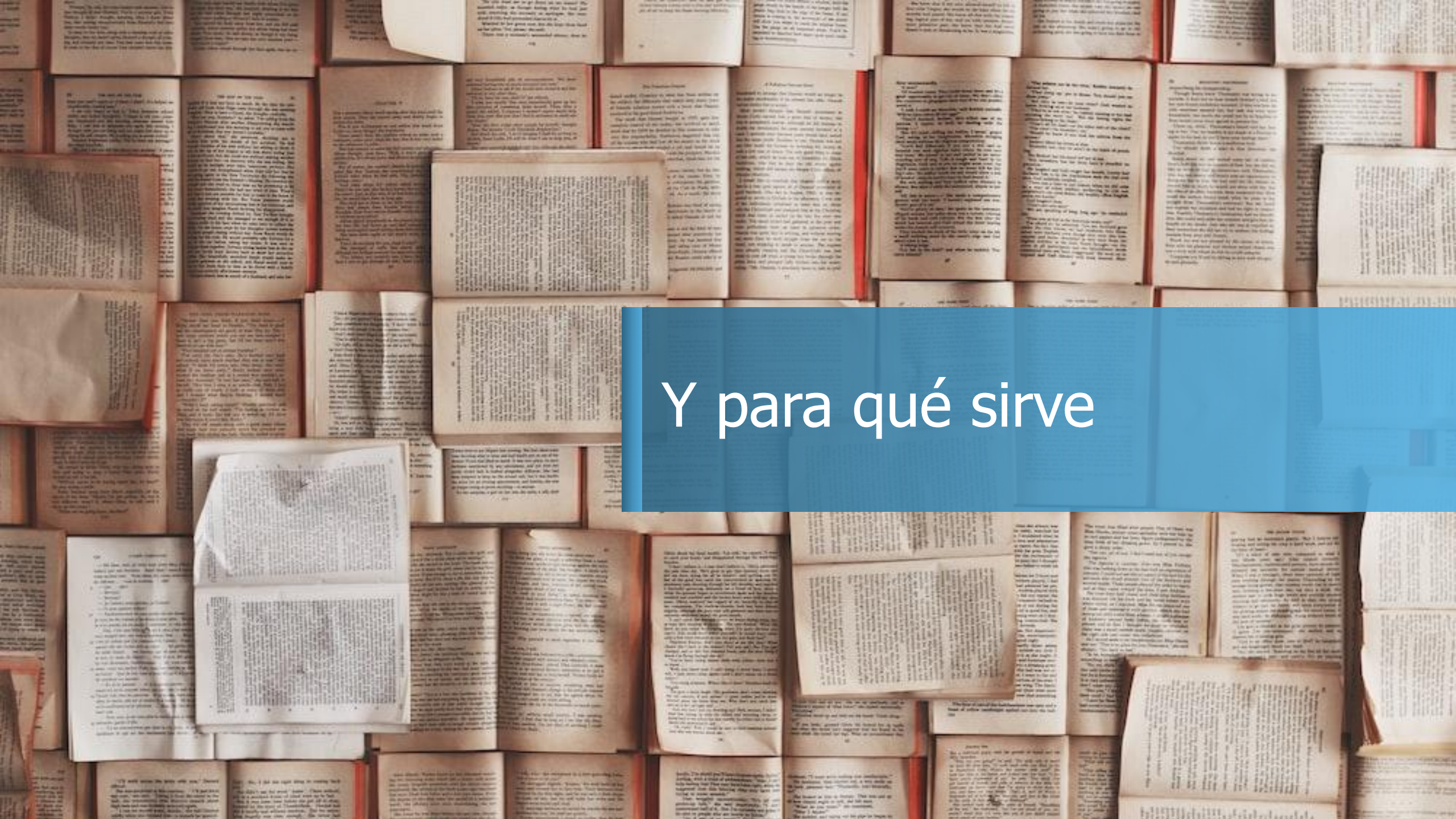
Representaremos los nodos del problema en la lista L e iteraremos con los pasos siguientes:

1. Sea c el valor artificial de la profundidad de corte (artificial breadth cutoff; anchura en la que se parará la búsqueda DFS). Inicialmente el valor es 2.
2. Sea L una lista con los nodos iniciales del problema.
3. Si L está vacía y $c=d$ entonces terminar.
4. Si L está vacía pero $c < d$ entonces incrementar c y volver al paso 2 (el algoritmo vuelve a empezar con una c diferente).
5. Si no, sea n el primer nodo de la lista L .
6. Si n es el nodo objetivo entonces parar y retornarlo conjuntamente con el camino desde el nodo inicial hasta él.
7. Si no, eliminar n de L .
8. Expandir n generando sólo los primeros c hijos. Añadir estos hijos al principio de L . Cada hijo estará etiquetado con el camino recorrido desde el nodo inicial hasta él.
9. Volver a 3

Búsqueda ciega – métodos iterativos

- Comparativa de los métodos (resultados) + coste computacional/memoria entre todos (base + iterativos en el problema del laberinto)

	Iteraciones	Profundidad final	Profundidad explorada
DFS	?	?	?
BFS	?		
IDFS	?		
IBFS	?		



Y para qué sirve

¿Para qué nos sirve?

DFS

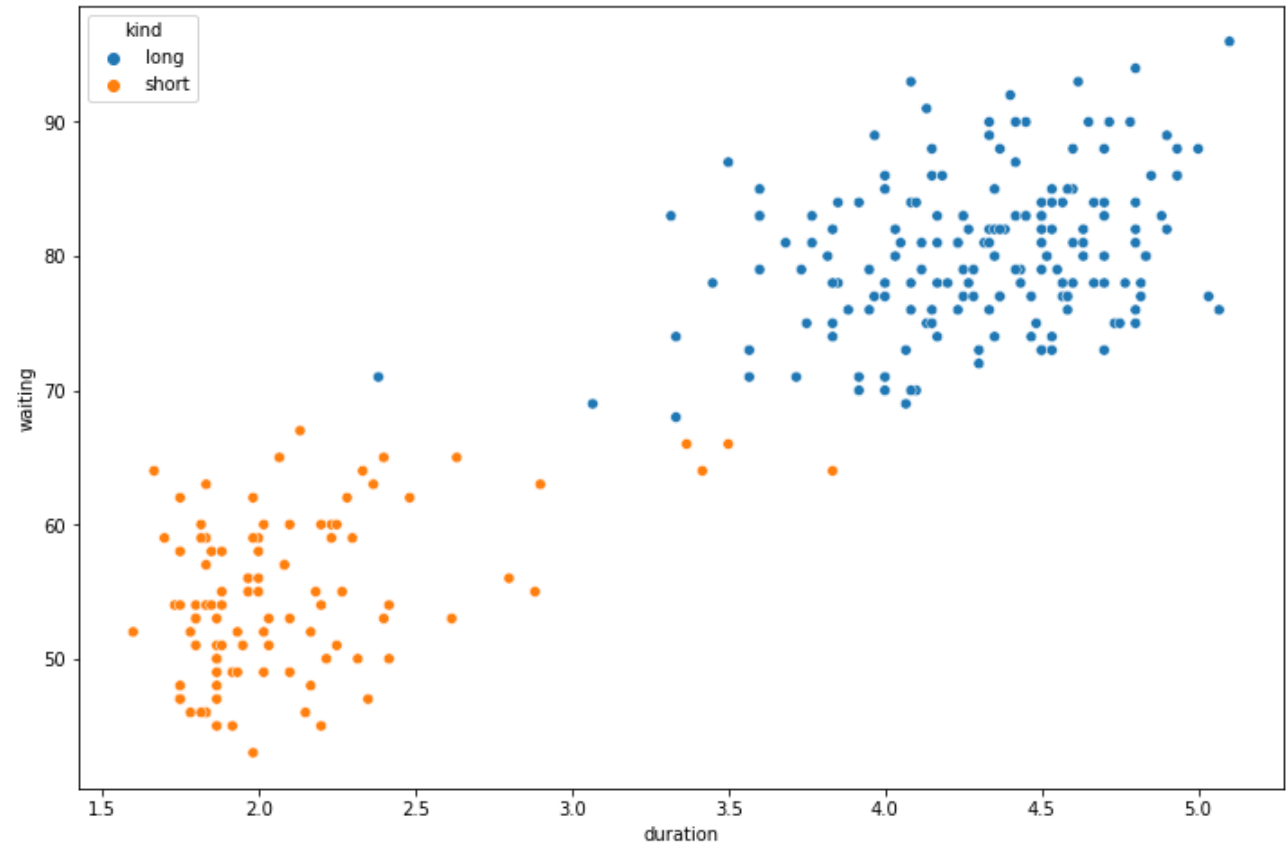
- Exploración laberintos
- Árbol de decisión
- Nodos y grafos
- Clasificación Topológica
- Movimientos de Juegos o resolver acertijos

BFS

- Encadenamiento de redes
- Conectividad
- RRSS
- Análisis de redes

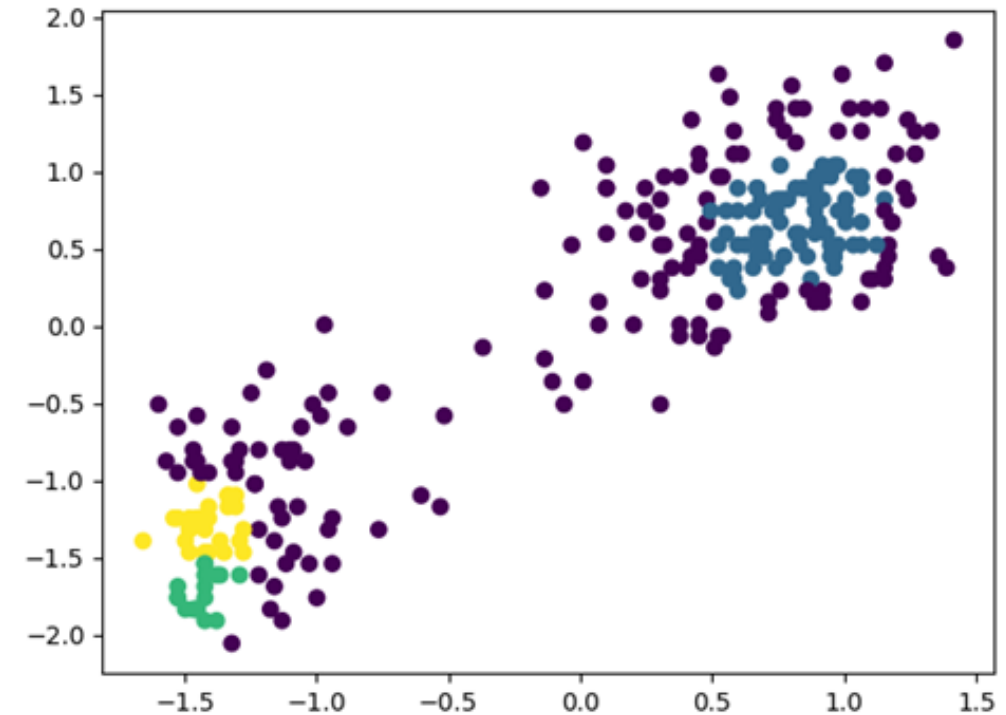
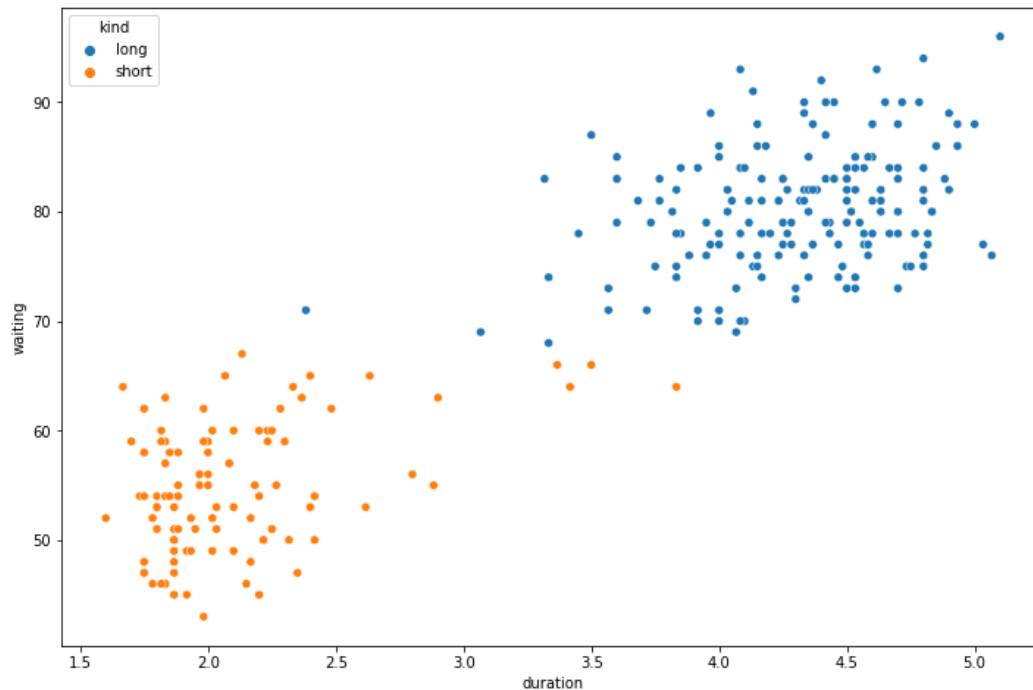
¿Para qué nos sirve?

- Atacar la casuística con la búsqueda de parámetros para 'futuros algoritmos' y comprender qué podemos y qué no podemos hacer dependiendo siempre del problema que tengamos delante.



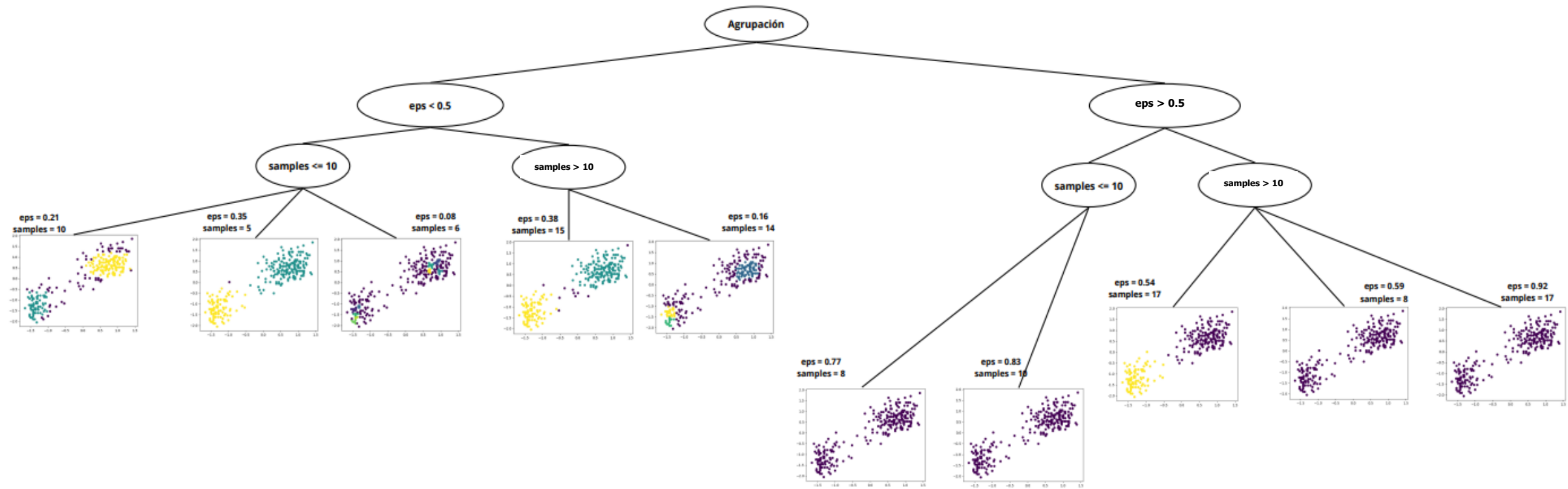
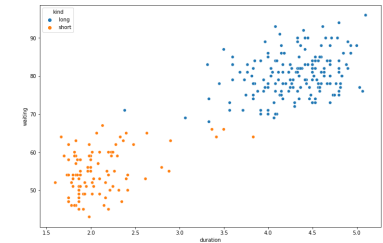
¿Para qué nos sirve?

Algoritmos de agrupamiento que necesitan un conjunto de parámetros para realizar su función de forma consistente, como por ejemplo **eps** (distancia entre puntos) y **samples** (número de muestras mínimas por agrupación).



¿Para qué nos sirve?

Utilizar este tipo de soluciones equivale a probar todas las combinaciones hasta encontrar un conjunto de parámetros funcional.





Manel Cerezo García – manel.Cerezo@salle.url.edu